大作业要求：

借助 DeepSeek 等大模型，开发包括但不限于工具类(文档编辑器、PDF 阅读器、格式转化工具、日程管理、饮食记录、旅行地图、记账等)、游戏类(贪吃蛇、俄罗斯方块、数独、消消乐、掼蛋等)等一款小程序/移动端 APP/桌面端软件。要求尽可能使用大模型设计、生成和调试代码，以不超过 4 人的小组为单位，展示和分享制作的小程序/移动端 APP/桌面端软件，着重分享使用大模型完成开发的心得和技巧，并以小组为单位提交大作业报告和材料。

DeepSeek 可用网站：http://chat.ustb.edu.cn （学校内网） https://yuanbao.tencent.com/

时间安排：

- 4 月 18 日（第七周周六）: 所有小组进行验收，包括PPT汇报和作品演示，重点是AI工具使用情况介绍和软件功能演示；
- 4 月 30 日：报告等所有材料提交截止。

评分标准(20)：

总分 20，其中验收演示共计 10分，报告和材料共计 10 分。

验收演示部分(10)：

- 开发平台选择(2)：小程序 2分，移动端 APP 2 分，桌面端 1 分；
- 代码复杂度和软件完成情况(5)：复杂度高且完成度高 5 分，复杂度低且完成度低 1 分，其他 2-4 分；
- 人工智能工具使用情况(3)：AI 工具种类和数量 0-1 分，AI 工具利用程度 0-2 分。

报告和材料部分(10)：

- 报告(8)：报告内容完整度 0-2 分，报告撰写质量 0-3 分，AI 工具使用和心得 0-3 分；报告中应包含团队成员具体分工、贡献比例及详细开发文档，说明和记录包括需求分析、系统设计、开发记录、测试、代码质量与版本控制(鼓励团队合作开发)、部署过程在内的完整开发流程；
- 材料(2)：完整代码(包括代码注释和说明等)和打包程序(小程序只需提交代码、移动端 APP 安装包，桌面端的可执行程序)。

大作业提交方式：

截止时间：2026-04-30 23:59
提交地址：贝壳学堂-作业-课程大作业

简单样例(仅供大模型的使用参考)：

如何用**Python**开发功能齐全的**PDF**阅读器

如何用Python开发一个功能齐全的PDF阅读器，要求实现打开PDF文件、页面跳转、文本搜索、缩放视图等核心功能，并可利用适合的库（如PyPDF2管理PDF文件、Tkinter或PyQt实现用户界面）进行开发，同时确保程序性能优化、易用性和多设备兼容性，并提供必要的开发和用户文档来支持使用和后续维护

已深度思考（用时50秒）

嗯，用户想要用Python开发一个功能齐全的PDF阅读器，需要实现打开文件、页面跳转、文本搜索、缩放视图这些核心功能，还要用PyPDF2和Tkinter或者PyQt。同时要注意性能优化、易用性、多设备兼容性，还要提供文档。好，首先我得考虑怎么分解这个任务。

首先，用户界面方面，Tkinter和PyQt都是选项。不过PyQt的功能更强大，界面更现代，可能更适合复杂的应用。但PyQt的许可问题需要注意，如果是商业用途可能需要购买许可，而Tkinter是Python自带的，但界面可能不够美观。用户可能更倾向于功能强大的PyQt，所以可能需要先确定这一点。不过用户提到"适合的库"，所以两种都可以选，但根据功能需求，PyQt可能更适合，特别是需要处理复杂的视图和缩放功能。

然后，PDF处理库，PyPDF2确实不错，不过最近有新的版本PyPDF3和PyPDF4，但可能PyPDF2更稳定。另外，还有pdfplumber这样的库，可能提取文本更准确。需要确认PyPDF2是否能满足所有需求，比如文本提取、页面跳转等。可能用户需要的是读取PDF内容并渲染到界面上，这时候可能需要渲染库，比如pdf2image将PDF转为图片显示，或者使用ReportLab生成预览，但可能需要结合其他库。或者用PyMuPDF（也就是fitz），它渲染PDF的效果不错，性能也好。但用户提到PyPDF2，可能需要结合其他库来处理渲染部分。

接下来，功能拆分：

1. 打开PDF文件：需要文件对话框选择文件，用PyPDF2读取PDF的元数据和页面内容，然后渲染第一页到界面。
2. 页面跳转：需要导航按钮（上一页、下一页）和输入页码跳转。这里需要跟踪当前页码，并更新显示。
3. 文本搜索：遍历PDF的每一页，查找关键字，高亮显示或者列出搜索结果。这里可能需要处理文本提取后的搜索，以及可能的性能问题，比如大文件时如何快速搜索。
4. 缩放视图：调整显示区域的大小，可能需要缩放图片或调整控件的大小。可能需要处理图像的缩放，比如用PIL库调整图片尺寸。

关于性能优化，可能需要缓存已渲染的页面图片，避免每次翻页都重新渲染。另外，使用多线程来处理耗时的操作，比如加载PDF或搜索文本，防止界面卡顿。

多设备兼容性可能需要考虑不同屏幕分辨率，界面布局的自适应，以及在不同操作系统（Windows、macOS、Linux）上的表现。PyQt本身在这方面处理得不错，但需要注意不同平台的UI差异。

用户文档需要包括开发文档（如何搭建环境、代码结构说明）和用户手册（如何使用各项功能）。可以用Sphinx生成文档，或者写Markdown文件。

现在，具体的实现步骤：

1. 确定技术栈：PyQt5用于UI，PyPDF2处理PDF元数据和文本提取，pdf2image或PyMuPDF用于渲染PDF页面为图像，因为PyPDF2本身不提供渲染功能，只能提取文本和元数据。所以可能需要另一个库来处理图像渲染。例如，使用pdf2image配合poppler将PDF页面转为图片，或者用PyMuPDF的fitz模块来渲染页面。
2. 设计UI布局：主窗口包含菜单栏（打开文件、退出）、工具栏（缩放按钮、搜索框）、页面导航控件（上一页、下一页、页码输入）、显示区域（可能是QLabel显示图像，或者自定义的视图控件），以及搜索结果的显示区域。
3. 实现打开文件功能：使用QFileDialog选择PDF文件，用PyPDF2加载PDF，获取总页数，初始化当前页码为1，然后渲染第一页。
4. 页面跳转：当用户点击上一页/下一页或输入页码时，更新当前页码，并重新渲染该页。需要处理页码的有效性（比如不能超过总页数）。
5. 文本搜索：当用户输入关键字并触发搜索时，遍历所有页面的文本，记录匹配的页面和位置。可能需要用PyPDF2提取每页的文本，然后使用字符串查找。对于大文件，可能需要后台线程处理，避免界面冻结。然后展示搜索结果，比如列表显示匹配的页面，点击后跳转到该页，并高亮文本。但高亮可能需要渲染层处理，比较困难，所以可能暂时只列出搜索结果，跳转到页面。
6. 缩放视图：用户选择缩放比例（如50%、100%、150%），调整显示图像的大小。可能需要存储原始图像的大小，然后根据缩放比例调整显示尺寸。使用QPixmap的scaled方法进行缩放，注意保持宽高比。

性能优化点：

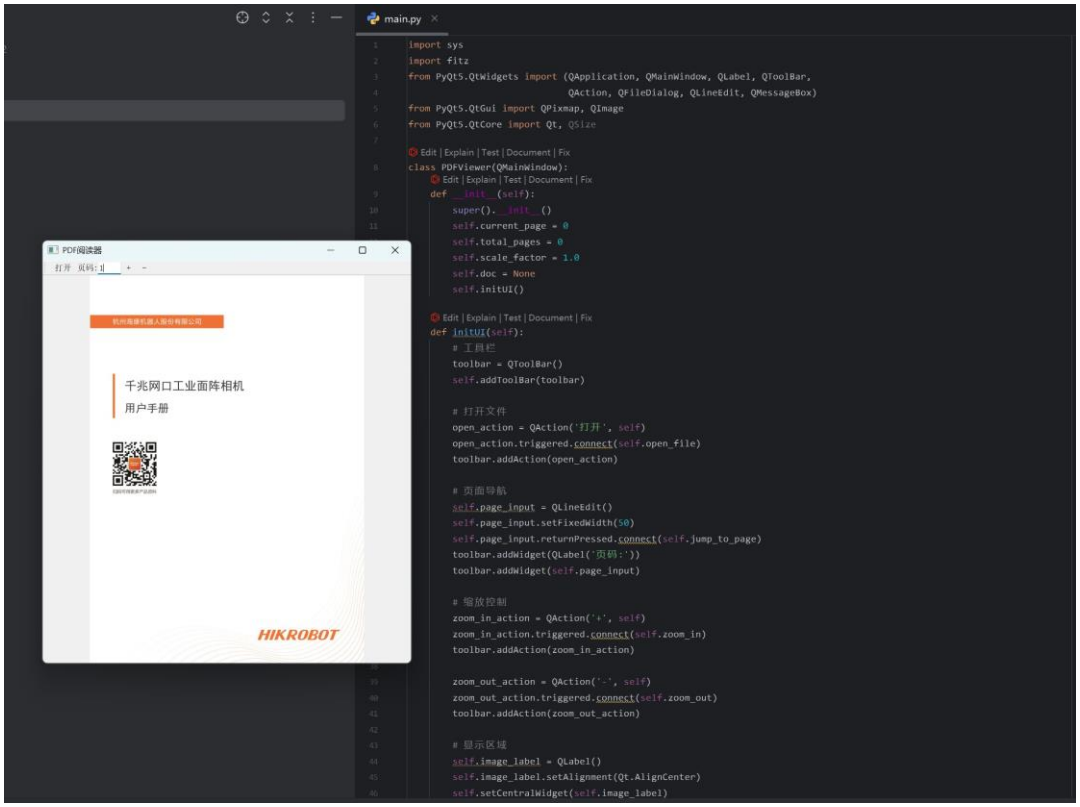
- 使用缓存：将已渲染的页面图像存储在内存或临时文件中，避免重复渲染。
- 异步加载：在后台线程中渲染页面，避免主线程阻塞。
- 延迟加载：当快速翻页时，取消未完成的渲染任务，只处理最新的请求。

多设备兼容性：

- 使用PyQt的布局管理器，确保界面在不同分辨率下自适应。
- 测试不同操作系统下的表现，特别是文件路径处理和字体显示。

可能的挑战：

- PDF渲染的质量和速度。PyMuPDF可能比pdf2image更f 安装可能依赖系统库，比如poppler。在Windows上可能需要预


main.py
import sys
import fitz
from PyQt5.QtWidgets import (QApplication, QMainWindow, QLabel, QToolBar,
QAction, QFileDialog, QLineEdit, QMessageBox)
from PyQt5.QtGui import QPixmap, QImage
from PyQt5.QtCore import Qt, QSize
Edit | Explain | Test | Document | Fix
class PDFViewer(QMainWindow):
Edit | Explain | Test | Document | Fix
def __init__(self):
super().__init__()
self.current_page = 0
self.total_pages = 0
self.scale_factor = 1.0
self.doc = None
self.initUI()
Edit | Explain | Test | Document | Fix
def initUI(self):
# 工具栏
toolbar = QToolBar()
self.addToolBar(toolbar)
# 打开文件
open_action = QAction('打开', self)
open_action.triggered.connect(self.open_file)
toolbar.addAction(open_action)
# 页面导航
self.page_input = QLineEdit()
self.page_input.setFixedWidth(50)
self.page_input.returnPressed.connect(self.jump_to_page)
toolbar.addWidget(QLabel('页码:'))
toolbar.addWidget(self.page_input)
# 缩放控制
zoom_in_action = QAction('+', self)
zoom_in_action.triggered.connect(self.zoom_in)
toolbar.addAction(zoom_in_action)
zoom_out_action = QAction('-', self)
zoom_out_action.triggered.connect(self.zoom_out)
toolbar.addAction(zoom_out_action)
# 显示区域
self.image_label = QLabel()
self.image_label.setAlignment(Qt.AlignCenter)
self.setCentralWidget(self.image_label)
PDF阅读器
打开 页码: 1
千兆网口工业面阵相机
用户手册
HIKROBOT